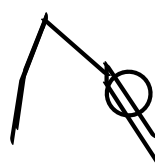
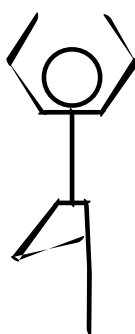


CLOSET

Three-dimensional articulated figures for Typst



Version 0.1.0

Contents

1. Overview	3
1.1. Installation	3
1.2. First figure	3
2. Three-dimensional model	3
2.1. Built-in poses	4
2.2. Controlled variation	4
3. Camera and projection	4
3.1. One pose, three views	5
3.2. Choosing a useful viewpoint	5
3.3. Framing and focal length	5
4. Calligraphic rendering	5
5. Imported OpenSim motion	6
6. API reference	6
6.1. human	6
6.2. vary-pose	6
6.3. camera	6
6.4. project	6
6.5. joints	6
6.6. pose-from-degrees and constrain-pose	6
7. Troubleshooting	6

1. Overview

Closet draws compact human figures from an articulated model. A pose contains three-dimensional bone directions. The package evaluates those directions into joint positions, projects the joints through a perspective camera, and sends the resulting paths to Premetadated's calligraphic pen engine.

The pipeline is:

```
3D pose directions
-> 3D forward kinematics
-> perspective camera
-> 2D projected paths
-> calligraphic nib envelopes
-> CeTZ vector output
```

This separation is important. Changing the camera does not alter the body or its pose; it only changes how the same 3D joints appear on the page.

1.1. Installation

For local package use, install both repositories under Typst's local package directory:

```
{data-dir}/typst/packages/local/closet/0.1.0
{data-dir}/typst/packages/local/premetadated/0.1.0
```

On macOS, {data-dir} is ~/Library/Application Support. Import the package as:

```
#import "@local/closet:0.1.0": human
```

From this repository, examples can instead use `#import "lib.typ"`.

1.2. First figure

```
#import "@local/closet:0.1.0": human

#human(pose: "running", scale: 1.2)
```

`human` returns a CeTZ canvas, so it can be placed in normal Typst grids, figures, boxes, and alignment containers.

2. Three-dimensional model

Closet uses right-handed (x, y, z) coordinates:

- x runs horizontally across a front-facing body;
- y carries depth, toward or away from the viewer;
- z points upward.

A canonical pose stores `torso` and `side` directions plus one direction for each upper and lower arm and leg. Directions need not be unit length; the kinematics normalizes them before applying fixed anatomical segment lengths.

```
#let custom = (
  torso: (0.0, 0.0, 1.0),
  side: (1.0, 0.0, 0.0),
  left-upper-arm: (0.7, 0.2, 0.7),
  left-lower-arm: (0.8, 0.1, 0.5),
  right-upper-arm: (-0.2, -0.8, -0.6),
  right-lower-arm: (-0.1, -0.9, -0.4),
  left-upper-leg: (0.2, 0.0, -1.0),
  left-lower-leg: (0.0, 0.0, -1.0),
```

```

    right-upper-leg: (-0.2, 0.0, -1.0),
    right-lower-leg: (0.0, 0.0, -1.0),
  )

  #human(pose: custom)

```

`joints(pose)` evaluates a pose without drawing it. It returns `pelvis`, `neck`, and `head` points plus three-point chains for each arm and leg. Every returned point is a 3D tuple.

2.1. Built-in poses

The package includes standing, sitting, cross-legged sitting, thinking, walking, running, downward dog, child's pose, warrior one, warrior two, tree pose, squatting, push-up, pull-up, walking on hands, jumping on one foot, and jumping with open legs. Their stable names are available as keys in `poses`; see `assets/previews/poses.png` for the visual index.

2.2. Controlled variation

`vary-pose(pose, variation:, seed:)` adds small deterministic offsets to the canonical 3D bone directions. It accepts either a built-in pose name or a pose dictionary and returns a new dictionary suitable for `human`, `joints`, or additional editing.

```

#let people = range(5).map(seed =>
  vary-pose("walking", variation: 0.09, seed: seed)
)

#for person in people {
  human(pose: person, camera: profile-camera)
}

```

The same operation is available directly on `human`:

```

#human(pose: "walking", variation: 0.09, seed: 3)

```

The integer `seed` makes output reproducible. Equal inputs produce identical figures, while different seeds redistribute each limb slightly. Variation is bounded to `0..0.35`; `0` returns the canonical pose, and `0.04..0.12` usually gives useful scene diversity without changing the gesture. Torso and body-side axes receive a smaller perturbation than limbs. All directions are normalized after perturbation, so bone lengths remain fixed.

3. Camera and projection

Create a perspective camera with:

```

#let view = camera(
  eye: (3.8, -8.0, 2.8),
  target: (0.0, 0.0, 0.2),
  up: (0.0, 0.0, 1.0),
  focal-length: 7.0,
)

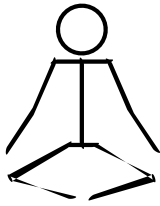
```

`eye` is the camera position. `target` is the world-space point at the center of the image. `up` establishes image vertical. `focal-length` controls perspective magnification: increasing it enlarges the projected figure when camera distance is unchanged.

The implementation constructs a look-at basis from these values. For point p , eye e , forward vector f , camera-right vector r , and camera-up vector u , depth is $d = (p - e) \cdot f$. Projection is proportional to $((p - e) \cdot \frac{r}{d}, (p - e) \cdot \frac{u}{d})$. Points must remain in front of the camera.

3.1. One pose, three views

The following three figures all use exactly `poses.sitting-cross-legged`. Only the camera position changes.



Front



Three-quarter



Side

The front view emphasizes bilateral symmetry. The three-quarter view reveals which knees and hands advance in depth. The side view compresses the body's width but exposes front-to-back placement. This is why a pose that looks ambiguous from one direction can become immediately legible from another.

```
#let pose = poses.sitting-cross-legged

#human(pose: pose, camera: camera(eye: (0, -8, 2.4)))
#human(pose: pose, camera: camera(eye: (5, -7, 3.0)))
#human(pose: pose, camera: camera(eye: (8, 0, 2.4)))
```

3.2. Choosing a useful viewpoint

Use a front view for poses whose defining feature is left-right symmetry, such as warrior two, tree pose, squatting, and jumping. Use a profile view for poses whose silhouette depends on forward motion or floor contact, such as sitting, walking, running, downward dog, child's pose, and push-up. A three-quarter view is useful when depth relationships matter but both sides must remain visible.

The supplied `front-camera`, `profile-camera`, and `three-quarter-camera` are convenient defaults, not semantic constraints. A pose may face another world direction, and a custom `camera` can always be more informative.

3.3. Framing and focal length

Move `eye` and `target` to frame the subject. Keep `target` near the torso for upright poses and lower it for floor poses. If the figure is too small, increase `focal-length`, increase `human(scale:)`, or move the eye closer. Changing focal length alters perspective and magnification; changing `scale` only changes the final drawing size and pen dimensions.

4. Calligraphic rendering

Every bone and the head outline are rendered by `premetadated.stroke.nib-stroke`. The default tangent-following nib has a broad axis of `0.050`, a fine axis of `0.015`, and an offset of `24deg`. Nib dimensions scale with the figure.

Pass any `Premetadated` pen dictionary to override it:

```
#human(
  pose: "yoga-warrior-two",
  pen: (
    mode: "calligraphic",
    offset: 12deg,
    samples: ((arclength: 0, a: 0.055, b: 0.009),),
  ),
  stroke: rgb("352b25"),
)
```

Larger separation between `a` and `b` creates stronger contrast between broad and fine directions. The `nib` offset rotates that contrast relative to each path's tangent.

5. Imported OpenSim motion

`tools/import_opensim.py` is an offline adapter. It downloads pinned Gait2354 model and inverse-kinematics data, then writes compact model ranges and paired gait frames to `data/gait2354.json`.

```
python3 tools/import_opensim.py
typst compile --root . examples/opensim.typ build/opensim.pdf
```

`pose-from-degrees(base, values)` converts imported sagittal hip and knee angles into 3D leg directions. Use a canonical 3D pose as the upper-body base:

```
#let data = json("data/gait2354.json")
#let sample = data.samples.first()
#let pose = pose-from-degrees(poses.walking, sample.pose_degrees)
#human(pose: pose, limits: data.limits, camera: profile-camera)
```

The synchronized hip and knee values belong to one observed time step. Joint ranges alone do not guarantee a realistic combined pose, and the Gait2354 ranges are model domains rather than universal physiological safety limits.

6. API reference

6.1. human

`human(pose:, variation:, seed:, scale:, stroke:, pen:, limits:, camera:)` renders one figure. `pose` accepts a built-in name or complete 3D pose dictionary. `stroke` is the ink color. `pen` accepts a `Premetadated` pen; `none` selects the package default. `limits` optionally clamps imported OpenSim leg angles. `camera` controls the projection.

6.2. vary-pose

`vary-pose(pose, variation:, seed:)` returns a reproducibly perturbed 3D pose. The accepted variation range is `0..0.35` and the seed must be an integer.

6.3. camera

`camera(eye:, target:, up:, focal-length:)` validates and returns a camera dictionary. The package exports `default-camera`, `front-camera`, `profile-camera`, and `three-quarter-camera`.

6.4. project

`project(point, camera:)` maps one 3D point to a 2D pair. It is useful when labels or other graphics must align with a figure joint.

6.5. joints

`joints(pose)` evaluates forward kinematics and returns the 3D body landmarks without rendering.

6.6. pose-from-degrees and constrain-pose

These helpers bridge compact OpenSim-derived data to the 3D renderer. They are primarily intended for generated sagittal gait samples.

7. Troubleshooting

- **Package not found:** install both `closet:0.1.0` and `premetadated:0.1.0` in the same local package namespace.
- **A pose looks flat:** move the camera away from a principal axis, for example from `(0, -8, 2.4)` to `(4, -7, 3)`.

- **A limb is hidden:** rotate the camera around **target**; do not modify the pose merely to compensate for one viewpoint.
- **The figure is clipped:** move **eye** farther away or reduce **scale**.
- **Projection reports a point behind the camera:** move **eye** away from the model and keep **target** between the camera and figure.
- **Calligraphy is too subtle:** increase nib axis **a** or reduce **b**.